

## **ExpenseTracker: Initial Planning and Design**

Group Project Deliverable 1 (Friday) - Option 1: Expense Tracker Application

Languages: Python and C++

Rahul Solanki, Krinal Soni, and Bijay Raj KC

University of the Cumberland

MSCS-632-M30: Advanced Programming Language

Jay Thom

October 2, 2026

## ExpenseTracker: Initial Planning and Design

### Project Overview

Our team is assigned Option 1, an expense tracker, implemented in Python and C++. The application, which we call ExpenseTracker, records expenses with a date, an amount, a category and a description, filters them by category and by date range, searches their descriptions, and summarises spending by category and overall. We will build it twice, once per language, with identical external behaviour.

The pairing is well suited to the purpose of the assignment because the two languages disagree on almost every design question this application raises. Python decides types while the program runs and manages memory for the programmer; C++ fixes types at compile time and makes ownership an explicit part of the code (Sebesta, 2019). An expense tracker is small enough to finish in three days and still exercises records, containers, indexing, dates, aggregation and error handling, so the differences show up in ordinary code rather than in contrived examples.

### Application Design

#### *Core Requirements*

**Table 1**

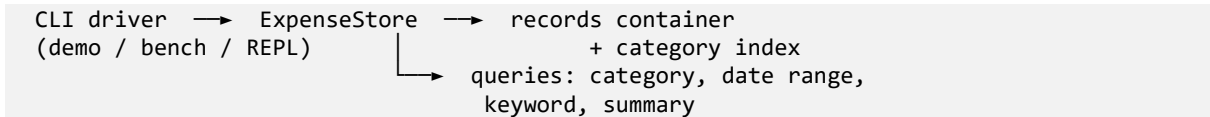
*Required Features and How the Design Satisfies Them*

| Requirement                                                           | Design element                                                                                                             |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Data storage for expenses with date, amount, category and description | A record carries id, date, amount, category and description; the store assigns the id and the validation rules are shared  |
| Filter and search by criteria such as date range and category         | by_category(name), in_range(from, to) and search(keyword); matching is case-insensitive and ranges are inclusive           |
| Summary of totals by category and overall                             | summary() returns per-category count, total and percentage share ordered by total, plus the overall total and record count |

## Architecture

Both implementations use the same two layers, so that differences in the code are attributable to the languages rather than to differing designs. A driver provides the command-line modes and a store holds the records and answers every query. The store keeps a category index alongside the records so that filtering by category is a lookup rather than a scan.

**Figure 1. Architecture shared by both implementations.**



**Table 2**

### *Components and Their Locations in Each Implementation*

| Component | Responsibility                            | Python          | C++                   |
|-----------|-------------------------------------------|-----------------|-----------------------|
| Record    | Build, validate and format one expense    | expense.py      | expense.hpp           |
| Store     | Hold records; run filters and the summary | store.py        | store.hpp / store.cpp |
| Driver    | CLI modes: demo, bench, interactive       | main.py         | main.cpp              |
| Tests     | Record helpers and store queries          | test_expense.py | tests.cpp             |

## Command-Line Interface

Both implementations expose the same three modes, which keeps demonstration and measurement symmetrical. The demo mode loads a seeded dataset of ten expenses and then runs every query in a fixed order, so the two outputs can be compared line by line. The bench mode inserts a configurable number of generated expenses and reports insert and query timings. With no arguments the program is an interactive prompt supporting /add, /list, /cat, /range, /search, /summary, /del and /quit.

## Anticipated Language Differences

Table 3 records what we expect before writing the code. The Day 3 comparison report revisits each row and states where we were right and where we were wrong, which is more useful than reporting only the predictions that happened to hold.

**Table 3***Expected Differences, Recorded Before Implementation*

| Area              | Expectation                                                                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Type systems      | Python will accept an amount given as an int, a float or a string and fail only when the bad value is used; C++ fixes the parameter type, so the same mistake cannot be written at all |
| Data structures   | Python dict and C++ <code>std::unordered_map</code> are both hash tables, so the algorithmic shape should match and only the constant factors should differ                            |
| Memory management | Python reference-counts and collects cycles; C++ containers own their buffers and free them in destructors, so we expect no new or delete anywhere and a smaller footprint             |
| Dates             | Python's datetime parses, validates and compares out of the box; C++20 chrono supplies a calendar type but we expect to write parsing and formatting ourselves                         |
| Error handling    | Both use exceptions, but Python will surface type problems at run time where C++ surfaces most of them at compile time                                                                 |
| Performance       | C++ faster on insert and on queries, probably by a single-digit multiple rather than orders of magnitude, because both use hash-indexed containers                                     |

***Risks Identified Up Front***

- Index invalidation. If the C++ store indexes records by their position in a vector, deleting a record shifts every later element and invalidates the index. Mitigation: rebuild the index on delete and document the cost against Python's id-keyed approach, which does not have the problem.
- Floating-point money. Binary floating point cannot represent every decimal amount exactly. Mitigation: both implementations use a double and format to two decimals, so they agree with each other; the limitation is recorded rather than hidden.
- Toolchain parity. Timings are comparable only if both run on one machine. Mitigation: a single WSL2 Ubuntu 24.04 environment with Python 3.12.3 and g++ 13.3.0 for every build and measurement.

**Task Assignment**

Roles are assigned so that each member owns one language or the cross-cutting analysis, and so that no member is blocked waiting on another. The shared specification is agreed first by

all three, which is what makes the two implementations comparable rather than merely similar.

All three members present on Sunday.

**Table 4**

*Role Assignment*

| Member        | Primary responsibility                             | Also contributes                                      |
|---------------|----------------------------------------------------|-------------------------------------------------------|
| Rahul Solanki | Python implementation: record, store, queries, CLI | Benchmark harness and measurement; repository setup   |
| Krinal Soni   | C++ implementation: struct, STL containers, CLI    | Unit tests; AddressSanitizer and warning-clean builds |
| Bijay Raj KC  | Shared specification and documentation             | Comparison report, screenshots, presentation assembly |

**Timeline**

**Table 5**

*Three-Day Milestone Plan*

| When   | Milestone                                                     | Owner             |
|--------|---------------------------------------------------------------|-------------------|
| Fri    | Shared specification, architecture and query semantics agreed | Bijay (lead), all |
| Fri    | Three repositories created; planning report submitted         | Rahul             |
| Sat AM | Record and store types compiling in both languages            | Rahul, Krinal     |
| Sat PM | Seeded dataset, listing and category filter working           | Rahul, Krinal     |
| Sat PM | Core-functionality report with screenshots                    | Bijay             |
| Sun AM | Date range, keyword search, shares, delete, interactive mode  | Rahul, Krinal     |
| Sun AM | Unit tests in both; AddressSanitizer run on the C++ build     | Krinal            |
| Sun PM | Benchmark runs and comparison report                          | Rahul, Bijay      |
| Sun PM | Presentation slides and rehearsal                             | All               |

**Repository**

The planning materials for this deliverable, including the full shared specification with the seeded dataset, the expected totals and the acceptance checklist, are in the public repository at <https://github.com/iknalos/MSCS-632-M30-Group-D1>. Implementation is tracked in separate repositories for Day 2 and Day 3, which are listed in that repository's README.

## References

Meyers, S. (2014). *Effective modern C++: 42 specific ways to improve your use of C++11 and C++14*. O'Reilly Media.

Python Software Foundation. (2026). *The Python language reference* (Version 3.14). Retrieved October 2, 2026, from <https://docs.python.org/3/reference/>

Sebesta, R. W. (2019). *Concepts of programming languages* (12th ed.). Pearson.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.